MASTER PROMPT — M&A DEAL HUB

1. ROLE AND OBJECTIVE

Act as a senior full-stack developer, financial data engineer, UX/UI designer and investment banking technology specialist.

Build a complete, professional, production-minded web application called **M&A Deal Hub**, a searchable global database of mergers and acquisitions transactions.

The application must allow users to discover, search, filter, compare and analyze real M&A transactions from recent years.

This must be a working application, not a static mockup, a landing page with non-functional buttons, or a visual prototype.

The mandatory development and deployment platforms are:

- GitHub: source code repository and version control.
- Vercel: hosting, builds, previews and production deployment.

Do not propose replacing either platform.

The final result must be visually polished, technically reliable, responsive and suitable for presentation as a university project related to finance, corporate strategy and M&A.

2. FIRST PRIORITY: PREVENT DEPLOYMENT FAILURES AND 404 ERRORS

Before creating the application, inspect the existing repository and determine its actual structure, framework, dependencies and configuration.

If the repository is empty, initialize a clean project. If it already contains files, preserve useful existing work and avoid creating conflicting configurations.

Use a supported, Vercel-compatible version of Next.js with TypeScript and the App Router.

Mandatory requirements:

- The production homepage must resolve at `/`.
- Every internal navigation link must point to a real implemented route.
- Use a consistent App Router directory structure.
- Ensure the root layout, homepage, stylesheets and required configuration files exist in the correct locations.

- Include a custom `not-found.tsx` page with a clear explanation and a working link back to `/`.
- Include appropriate error handling and useful fallback states for failed data requests.
- Ensure the production build does not depend on development-only files, local absolute paths or uncommitted assets.
- Avoid unnecessary custom rewrites, redirects and static-export settings.
- Do not configure static export unless it is explicitly needed and compatible with all selected features.
- Do not hardcode localhost URLs in production navigation or API requests.
- Verify that the root route and every implemented route resolve correctly.
- Ensure that refreshes and direct visits to nested routes work.
- Avoid importing missing modules, using nonexistent components or referencing unavailable images and environment variables.
- Do not leave placeholder imports, TODOs that break execution, or unfinished routes.
- Provide valid `package.json` scripts, including `dev`, `build` and `start`.
- Use compatible dependency versions and a valid lockfile when package installation is supported.
- Never claim that a build, test or deployment succeeded unless it was actually executed and its result verified.

Before considering the application complete, run the production build and available automated tests. Fix all build errors and critical warnings that can reasonably be resolved.

If the available environment cannot run a build or access Vercel, clearly identify the remaining manual verification steps rather than claiming the deployment has been tested.

3. TECHNOLOGY STACK

Use:

- Next.js with App Router.
- React.
- TypeScript with strict type checking.
- Tailwind CSS.
- A consistent, reusable component system.
- Recharts for charts where useful.
- Lucide icons for interface icons.
- A validated data-access layer for all transaction data.
- A database or external data API only when required and properly configured.

Keep the architecture as simple as possible while maintaining professional quality.

Avoid adding unnecessary frameworks, packages or infrastructure.

The application must be deployable through a GitHub repository imported into Vercel, with automatic deployments on subsequent commits.

4. PRODUCT VISION

Create an M&A intelligence platform inspired by the usability of professional financial research terminals and the visual restraint of institutional investment banking websites.

The product should feel credible, analytical, premium and data-driven.

Brand name: M&A Deal Hub.

Suggested tagline: “Discover the deals shaping global business.”

The application should prioritize the transaction database and search experience over marketing content.

The interface must not look like a generic AI-generated dashboard template.

5. VISUAL DESIGN SYSTEM

Use a premium institutional finance aesthetic.

Color palette:

- Deep navy for the primary navigation and important backgrounds.
- White and very light grey for content surfaces.
- Dark charcoal for primary text.
- Muted grey for secondary information.
- Restrained emerald green for positive metrics and completed transactions.
- Amber for pending transactions.
- Red only for cancelled transactions or meaningful negative indicators.

Typography:

- Modern, highly legible sans-serif.
- Strong typographic hierarchy.
- Clear transaction values and financial metrics.
- Consistent spacing and alignment.

Design principles:

- Clean layouts with generous whitespace.
- Refined borders and subtle shadows.
- Compact but readable transaction tables.
- Consistent button, input, badge and card styles.
- Restrained animations.
- Professional loading skeletons.
- No excessive gradients, decorative effects or oversized rounded cards.
- No fake terminal graphics or fabricated statistics.

The application must work well on desktop, tablet and mobile.

6. REQUIRED PAGES

Implement all of the following routes.

`/` — Home and search

Create a professional homepage containing:

- Brand and navigation.
- A prominent global transaction search bar.
- Short explanation of the platform.
- Quick search suggestions.
- Recently announced or updated transactions when verified data is available.
- Featured transactions with sourced deal values.
- Quick filters for sector, announcement year, status and transaction size.
- A clear entry point to the full transaction database.

Do not display fabricated live market statistics.

If data is unavailable, display an honest empty state and a clear explanation.

`/deals` — Transaction database

This is the core product.

Implement a searchable and filterable transaction explorer.

Each transaction record should support, where available:

- Acquirer.
- Target company.
- Announcement date.
- Completion date.
- Transaction value.
- Currency.
- Transaction status.
- Industry and sector.
- Acquirer and target country.
- Transaction type.
- Deal description.
- Source links.

Provide:

- Full-text search.
- Filtering by announcement date and year.
- Filtering by sector.
- Filtering by country or region.

- Filtering by transaction status.
- Filtering by transaction value.
- Sorting by announcement date and deal value.
- Pagination or a reliable load-more mechanism.
- Clear-all-filters functionality.
- URL query parameters for shareable searches.
- Loading, empty, error and retry states.

Search must match the acquirer and target independently. Searching for a company should find transactions where it was either the buyer or the acquired business.

Clearly distinguish announcement date from completion date.

Never interpret an unknown transaction value as zero.

`/deals/[id]` — Transaction detail

Create a dedicated page for each transaction.

Include:

- Transaction overview.
- Acquirer and target.
- Deal value and currency.
- Announcement and completion dates.
- Transaction status.
- Industry and geography.
- Transaction rationale, when supported by reliable sources.
- Key milestones, when known.
- Source links and publication dates, when available.
- Related transactions, when meaningful.

Use stable transaction identifiers or validated slugs.

Unknown identifiers must trigger a controlled not-found experience, not an application crash.

`/analytics` — Market analysis

Provide useful analytical charts based exclusively on the dataset actually available.

Possible charts:

- Transactions by announcement year.
- Deal count by sector.
- Disclosed transaction value by year.
- Deal status distribution.
- Largest transactions in the available dataset.

Every chart must use real loaded data and handle empty datasets.

Disclose that totals based on undisclosed values are incomplete.

Do not present sample data as actual market-wide statistics.

`/about` — Methodology and sources

Explain:

- What qualifies as an M&A transaction.
- The difference between announcement and completion.
- How transaction values and currencies are represented.
- The origin of the dataset.
- Known coverage limitations.
- How users can verify a transaction.

Include a clear disclaimer that the product is an informational research tool, not investment advice.

7. DATA STRATEGY — CRITICAL

A professional search interface is only useful if the underlying data is reliable.

Design a modular data provider so the user interface does not depend on one specific data vendor.

Use this order of preference:

1. A legitimate M&A or financial data API with documented access and appropriate usage rights.
2. Public official disclosures and other permitted, verifiable sources.
3. A clearly labelled, curated seed dataset for initial development and demonstration.

Before integrating any external provider, check its documentation, authentication method, query limits, coverage, pricing, attribution requirements and commercial restrictions.

Do not assume that a free API provides comprehensive global M&A coverage.

Do not scrape restricted databases, bypass authentication or invent API endpoints.

Never fabricate a transaction, transaction value, announcement date, completion date or source.

When the full provider is not configured, allow the application to operate using a small, clearly labelled demonstration dataset.

Show an explicit indication that demonstration records are examples and not live or comprehensive market data.

Build a typed data-provider interface so a future provider can be integrated without rewriting the search interface.

Include a seed dataset containing a small number of well-documented, historically significant transactions. Verify each included record against reliable sources and distinguish announced values from completed transaction values where relevant.

If a fact cannot be verified, mark it as unknown rather than guessing.

8. SEARCH ENGINE AND PERFORMANCE

Implement search with an architecture appropriate to the dataset size.

For the initial version, local search over the curated dataset is acceptable.

For a larger dataset, use server-side filtering and pagination through an authorized API or database.

Requirements:

- Case-insensitive search.
- Matching against acquirer, target and company aliases where available.
- Debounced text input where appropriate.
- Stable sorting.
- Filter combinations that work together.
- Useful feedback when no results match.
- No unnecessary loading of the entire database into the browser.
- No exposure of private API credentials.
- Clear handling of rate limits and network errors.

Do not claim universal coverage unless the connected data provider actually supports it.

9. SECURITY AND ENVIRONMENT VARIABLES

Keep private API keys on the server.

Use environment variables and document them in `.env.example` with placeholder values only.

Never commit real credentials.

Never prefix secret server-side credentials with `NEXT_PUBLIC_`.

Validate incoming query parameters and API responses.

Do not expose raw stack traces, private configuration or sensitive error information to users.

If an API key is missing, show a controlled, informative message and preserve the ability to use the demonstration dataset where appropriate.

10. USER EXPERIENCE

Every visible control must perform its advertised action.

Implement:

- Functional navigation.
- Search and filters.
- Sorting.
- Pagination.
- Working transaction detail links.
- Reset filters.
- Retry on recoverable data errors.
- Responsive navigation.
- Loading indicators.
- Accessible keyboard interactions.
- Visible focus states.
- Semantic HTML.
- Appropriate labels for form fields.
- Useful screen-reader descriptions where needed.

Do not create buttons that do nothing.

Do not show fake success messages.

Use consistent dates, currency formatting and number formatting.

11. ACCESSIBILITY, SEO AND QUALITY

Include:

- Page titles and metadata.
- Descriptive headings.
- Accessible color contrast.
- Keyboard-accessible controls.
- Appropriate language metadata.
- Useful descriptions for chart content.
- Clean URL structures.
- A sensible favicon and site identity.
- Responsive layouts without horizontal overflow.
- Reasonable image sizes and performance optimization.

Do not make the product dependent on remote decorative images to render correctly.

Use CSS, local assets or graceful fallbacks when external images are unavailable.

12. REQUIRED TESTS

Create automated tests for critical functionality where practical.

At minimum, verify:

1. The homepage renders.
2. `/deals` renders.
3. Search returns matching acquirers.
4. Search returns matching targets.
5. Multiple filters combine correctly.
6. Sorting and pagination behave correctly.
7. A valid transaction ID opens its detail page.
8. An invalid transaction ID produces a controlled not-found page.
9. Missing optional data does not crash rendering.
10. Failed API requests show an error state and retry option.
11. Empty search results are handled correctly.
12. Unknown transaction values do not appear as zero.
13. Production compilation succeeds.
14. Internal links do not point to nonexistent routes.
15. Refreshing or directly opening implemented nested routes works in the intended deployment configuration.

If browser-based end-to-end testing is available, test the production application locally before deployment.

Never mark unexecuted tests as passed.

13. GITHUB AND VERCEL DEPLOYMENT

The project must be prepared for a straightforward deployment using GitHub and Vercel.

Document these steps in a clear `README.md`:

1. Install the required runtime and dependencies.
2. Run the project locally.
3. Run tests and the production build.
4. Create or use the GitHub repository.
5. Push the project to GitHub.
6. Import the repository into Vercel.
7. Verify the framework preset and root directory.
8. Configure any required environment variables.
9. Deploy to production.
10. Verify the production URL and the main routes.
11. Review build logs if a deployment fails.
12. Make a later GitHub commit and verify that it triggers the expected deployment.

Use Vercel's standard Next.js integration rather than adding unnecessary deployment configuration.

Do not introduce a custom `vercel.json` unless a documented requirement makes it necessary.

If the repository is in a subdirectory or monorepo, explicitly configure and document the correct Vercel root directory.

14. CUSTOM ERROR AND NOT-FOUND EXPERIENCES

Build a polished custom 404 page that matches the application's design.

Include:

- A clear “Page not found” heading.
- A concise explanation.
- A working button or link to the homepage.
- A working link to the transaction database.

Add appropriate error boundaries and useful fallback UI for unexpected rendering errors.

Distinguish between:

- A missing route.
- An unknown transaction.
- A failed API request.
- A build or deployment failure.

Do not use a blanket rewrite to the homepage to conceal missing routes.

A custom 404 page improves the experience but does not fix a misconfigured deployment. Verify the actual root route and production configuration.

15. DELIVERY REQUIREMENTS

Build the actual application, not merely a plan.

Proceed in small, verifiable stages:

Stage 1: Inspect the repository and establish the correct project structure.

Stage 2: Implement the shared design system, root layout and homepage.

Stage 3: Implement the transaction data model, seed dataset and search interface.

Stage 4: Implement transaction detail pages and analytics.

Stage 5: Add data-source integration points, error handling, accessibility and documentation.

Stage 6: Run tests, perform a production build, fix discovered issues and review deployment compatibility.

Stage 7: Provide concise instructions for connecting GitHub and deploying through Vercel.

If you cannot complete a stage because credentials, data access or execution tools are unavailable, explain the limitation and continue with a working fallback where possible.

Do not replace working features with mockups merely to finish faster.

Do not claim that the application is live until its production deployment has been verified.

16. FINAL ACCEPTANCE CRITERIA

The project is complete only when:

- The homepage works.
- The transaction database works with the available dataset.
- Search, filters and sorting work.
- Transaction detail pages work.
- Analytics reflect the loaded dataset.
- Unknown routes produce a controlled 404 page.
- Errors and missing data are handled gracefully.
- The application passes the available tests.
- The production build succeeds.
- The GitHub repository structure is correct.
- The Vercel deployment configuration is documented.
- Required environment variables are documented.
- The README explains how to reproduce and deploy the project.
- No fabricated live data or unverified claims are presented as facts.

Start by inspecting the current repository and identifying the safest implementation path. Then build the application and verify each stage before proceeding.